AI RAG Waterproofing Chatbot

How to Run and Test the Project (Terminal CLI)

Generated: November 10, 2026

1. Overview

This project is a Retrieval-Augmented Generation (RAG) chatbot specialized in waterproofing and construction materials. It runs entirely from the terminal through backend/cli.py - no web server, frontend, or browser is required.

- Ingest documents into a local ChromaDB vector store.
- Ask questions in natural language (English or French).
- Answers are grounded in the retrieved document chunks, with sources printed in the terminal.

2. Prerequisites

- Python 3.10 or newer (check with: python --version)
- pip package manager
- About 3-4 GB free disk space for the first run (local models are downloaded automatically)
- Optional: an OpenAI or Anthropic API key if you prefer not to run models locally

3. Installation (Windows PowerShell)

Open PowerShell, go to the project folder, and run:

```
cd "D:\PROJECTS\AI RAG Waterproofing Chatbot"
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
copy config\.env.example config\.env
```

If PowerShell blocks the venv activation script, run this once: Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser

config/.env is only needed for the openai or anthropic providers. The default local provider needs no API key.

4. Configuration

All settings live in config/config.yaml. The most useful sections:

- embedding.provider: huggingface (default, runs locally) or openai
- llm.provider: local (default), openai, or anthropic
- llm.model: microsoft/phi-2 for the default local model
- rag.top_k_retrieval: how many chunks are retrieved per question (default 5)
- rag.chunk_size / rag.chunk_overlap: text splitting settings

If you use OpenAI or Anthropic, put your keys in config/.env:

```
OPENAI_API_KEY=your_openai_api_key_here
ANTHROPIC_API_KEY=your_anthropic_api_key_here
```

5. First Run - Ingest Documents

1. Put your source documents in data/processed/. Supported formats: PDF, DOCX, TXT, MD, CSV, XLS, XLSX.

2. From the project root, check the store and ingest:

```
python backend/cli.py stats
```

```
python backend/cli.py ingest -y
python backend/cli.py stats
```

Expected result: the chunk count goes from 0 to a positive number.

Re-ingesting the same files adds duplicate chunks. To start clean, delete the data/vector_store/ folder and ingest again.

6. Chatting

Interactive chat (default command):

```
python backend/cli.py
```

Ask a single question and exit:

```
python backend/cli.py ask "What is Sikalastic-560?"
```

Show the retrieved chunk text as well:

```
python backend/cli.py ask "Comment appliquer SikaTop-141 ?" --show-chunks
```

Example session:

```
You: Quelle est la consommation de Sikalastic-560 ?

Assistant: ...

Sources:
  [1] data/processed/sika_tunisie/Fiche Technique/Sikalastic-560.PDF, chunk 2 (relevance: 0.87)

You: exit
Goodbye!
```

In-chat commands:

- /help - list commands
- /sources on|off - show or hide retrieved sources
- /chunks on|off - show or hide retrieved chunk text
- /rag on|off - toggle retrieval (off = plain LLM answer)
- /stats - vector store statistics
- exit, quit, Ctrl+C or Ctrl+D - leave the chatbot

7. CLI Reference

- chat - interactive chat session (this is the default when no command is given)
- ask "question" - answer one question, then exit
- ingest [path] - load a file or directory into the vector store (default: data/processed)
- stats - show vector store statistics

Useful options:

- --no-sources - hide retrieved sources
- --show-chunks - print retrieved chunk text
- --no-rag - answer with the LLM only, no retrieval
- -y / --yes (ingest) - skip the confirmation prompt

8. How to Test the Project

Run this checklist from the project root:

- 1. python backend/cli.py stats - expect 0 chunks before ingestion.
- 2. python backend/cli.py ingest -y - expect 'Processing Complete' with 0 failed files.

- 3. python backend/cli.py stats - expect the chunk count to have increased.
- 4. python backend/cli.py ask "Your test question" --show-chunks - expect an answer plus a Sources list with file paths and relevance scores.
- 5. python backend/cli.py - start interactive chat, try /help, /sources off, /rag off, then exit.
- 6. pytest - the test suite is currently empty, but the command should run without import errors.

Pass criteria: the answer is grounded in the retrieved chunks, sources are shown with relevance scores, and the CLI exits cleanly.

9. Troubleshooting

- ModuleNotFoundError or missing packages: recreate the venv (python -m venv venv), activate it, and run pip install -r requirements.txt again.
- 'api_key client option must be set': set OPENAI_API_KEY or ANTHROPIC_API_KEY in config/.env, or switch llm.provider to local in config/config.yaml.
- 'the vector store is empty' warning: run python backend/cli.py ingest -y.
- First run is very slow: the local embedding/LLM models are downloading (~3 GB). Wait for it to finish or switch to an API provider.
- PowerShell activation error: run Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser once.